



SOLID и Clean Architecture

Пословно језгро остаје стабилно док се технички детаљи мењају

SOLID није формалност већ контрола промене

Принципи помажу да измене буду локализоване, зависности заменљиве и понашање проверљиво.

S

Single Responsibility

један кохерентан разлог за промену

O

Open/Closed

проширење без измене стабилног тока

L

Liskov Substitution

заменљивост без нарушавања очекивања

I/D

ISP и DIP

мали уговори и зависност од апстракција

OOP и Clean Code су основа SOLID-а

Пре принципа морају бити јасни уговор, енкапсулација, полиморфизам и одговорност.

ПОЈМОВИ

Interface дефинише очекивано понашање.

Енкапсулација чува инваријанте.

Полиморфизам омогућава заменљиве имплементације.

Апстрактна класа има смисла када постоји стварна заједничка основа.

ПРАКТИЧАН ЕФЕКАТ

код описује намеру

одговорност је видљива

промене су локализоване

тестови лакше прате понашање

Уговори пословног језгра из Logger–Blogger примера

Domain дефинише шта систем уме да ради, без знања о конкретном складишту или UI-ју.

```
1 public interface IZapisiServis
2 {
3     Result<ZapisNaSajtu> Objavi(ZapisNaSajtu zapis);
4     Result<IReadOnlyList<ZapisNaSajtu>> PregledZapisa();
5 }
6
7 public interface IBazaPodataka
8 {
9     TabeleBazaPodataka Tabele { get; }
10    bool SacuvajPromene();
11 }
```

Шта је важно

Уговор описује стабилну потребу пословног језгра.

Domain не зависи од Database или Presentation пројекта.

Заменљивост долази из јасних уговора, не из назива слојева.

SRP пита за разлог промене

Класа може имати више метода ако све припадају истој одговорности.

Проблем није број линија кода, већ чињеница да се иста класа мења из више међусобно независних разлога.

КЉУЧНА ПОРУКА

SRP усмерава дизајн ка кохерентним компонентама.

SRP y Logger–Blogger сервису

Сервис за записе оркестрира објаву, али не преузима UI, базу и логовање као једну одговорност.

```
1 public sealed class ZapisiServis : IZapisiServis
2 {
3     private readonly IBazaPodataka _baza;
4     private readonly ILogger _logger;
5
6     public Result<ZapisNaSajtu> Objavi(ZapisNaSajtu zapis)
7     {
8         _baza.Tabele.Zapisi.Add(zapis);
9         _baza.SacuvajPromene();
10        _logger.Info("Objavljen zapis");
11        return Result<ZapisNaSajtu>.Ok(zapis);
12    }
13 }
```

Шта је важно

Сервис оркестрира сараднике кроз уговоре.

Начин чувања и логовања остаје спољни детаљ.

Ако се промени logger, пословни ток не мора да се мења.

ОСР контролише познату варијабилност

Не уводи се апстракција за сваку хипотетичку промену.

Ако стабилан део система стално добија нови switch по типу понашања, променљиво понашање треба сместити иза уговора.

КЉУЧНА ПОРУКА

Циљ је проширење без измене централног тока.

ОСР кроз заменљиво складиште

Нови provider базе додаје се као нова имплементација уместо промене пословног кода.

```
1 public interface IBazaPodataka
2 {
3     TabeleBazaPodataka Tabele { get; }
4     bool SacuvajPromene();
5 }
6
7 public sealed class InMemoryBazaPodataka : IBazaPodataka
8 {
9     public TabeleBazaPodataka Tabele { get; } = new();
10    public bool SacuvajPromene() => true;
11 }
```

Шта је важно

Стабилан сервис користи IBazaPodataka.

JSON, in-memory или други provider су технички детаљи.

Проширење се дешава додавањем имплементације.

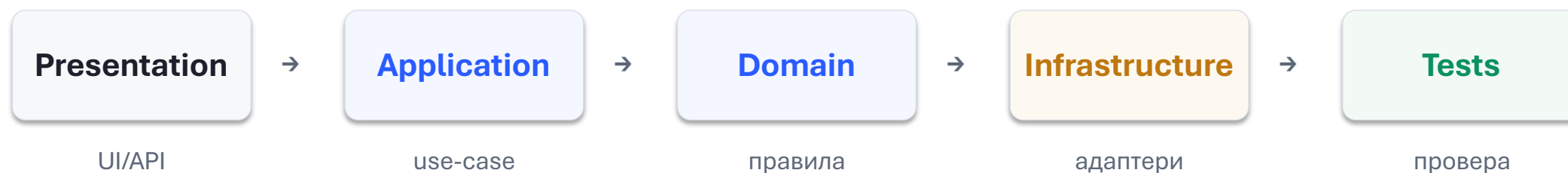
LSP, ISP и DIP исправљају границе

Ова три принципа најчешће откривају лоше уговоре и погрешан смер зависности.

Принцип	Проблем	Смер корекције
LSP	подтип мења очекивање	редизајнирати уговор
ISP	клијент зависи од вишка операција	поделити улоге
DIP	use-case креира SQL или HTTP детаљ	зависити од уговора

Clean Architecture је дисциплина граница

Називи пројеката могу да се разликују, али смер зависности мора бити јасан.



КЉУЧНА ПОРУКА

Пословно језгро не зависи од framework-а, базе, HTTP-а нити UI детаља.

Одговорности слојева морају бити објашњиве

Број пројеката није метрика квалитета архитектуре.

01

Domain

ентитети, инваријанте и стабилна правила

02

Application

случајеви употребе и портови

03

Infrastructure

база, датотеке и спољни сервиси

04

Presentation

превод захтева у позив use-case-a

Composition root из Logger–Blogger примера

Једно место бира конкретне имплементације, док пословни код остаје везан за уговоре.

```
1 IBazaPodataka baza = new JsonBazaPodataka("data.json");
2 ILogger logger = new FileLogger("app.log");
3
4 IZapisiServis zapisi = new ZapisiServis(baza, logger);
5 IConsoleIO io = new ConsoleIO();
6
7 var app = new AppContainer(zapisi, io);
8 app.Run();
```

Шта је важно

Presentation саставља граф зависности.

Domain и сервиси не креирају JSON базу или FileLogger.

DIP омогућава замену детаља без промене језгра.

Logger–Blogger је студија случаја

Циљ није преписивање шаблона, већ разумевање одговорности.

01

Пословно језгро

правила без
инфраструктурних детаља

02

Adapter-и

конкретна база, датотеке,
логовање

03

Composition root

место где се саставља граф
зависности

Refactoring мора имати разлог

Декомпозиција није циљ сама по себи.

ЛОШ СИГНАЛ

класа мења податке, шаље email и логује

метода има више неповезаних праваца

тест мора да зна приватне детаље

ново понашање тражи измену стабилног центра

ДОБАР СИГНАЛ

једна реченица описује одговорност

сарадници су уговори

промена погађа очекивано место

тестови проверавају понашање

Преглед кода кроз SOLID питања

Питања су кориснија од механичког препознавања принципа.

Питање	Шта открива
Да ли модул има један разлог за промену?	SRP
Да ли нова варијанта мења стабилан ток?	OCP
Да ли имплементација поштује уговор?	LSP
Да ли је интерфејс шири него што клијенту треба?	ISP
Да ли политика зависи од техничког детаља?	DIP

Рад на вежби

Циљ је да се принципи примене на конкретан код, не само дефинишу.

- 01 Пронаћи класу са најмање две неповезане одговорности.
- 02 Описати њену тренутну улогу једном реченицом.
- 03 Предложити раздвајање и нове границе.
- 04 Образложити које будуће промене постају локализоване.
- 05 Проверити да ли је смер зависности остао исправан.

Главни критеријум

SOLID се види кроз последице у коду.

1

једна јасна промена

Када се промени једно пословно правило или технички детаљ, треба да се мења мали и очекиван део система.

Најчешће грешке

Принципи могу бити погрешно примењени ако се претворе у шаблон.

01

Превише интерфејса

апстракције без реалне варијабилности

02

Погрешно наслеђивање

подтип не поштује уговор

03

Anemic domain

сва правила су у сервисима

04

Framework у језгру

Domain зависи од инфраструктуре

Суштина Вежбе 2

SOLID и Clean Architecture су начин да се заштити разумљивост система.

Добар дизајн није онај који има највише слојева, већ онај у коме се одговорност, смер зависности и последица промене могу јасно објаснити.

КЉУЧНА ПОРУКА

Следећи корак је смештање пословне логике у use-case и Domain слој.

За наредни термин

Применити принципе на пројектну целину.

- 01 Изабрати један use-case у свом домену.
- 02 Означити Domain, Application и Infrastructure одговорности.
- 03 Пронаћи једну класу са нејасном одговорношћу.
- 04 Предложити refactoring са образложењем.
- 05 Припремити пример за code review.